

Расширение возможностей векторизатора clang путём реализации оптимизационного пасса

УДК: 004.4'422

Научный руководитель: Курносов М. Г.

Проведён анализ способности автоматической векторизации циклов из тестового набора Extended Test Suite For Vectorizing Compilers (ETSVC) компилятором Clang/LLVM для архитектуры RISC-V RV64GCVB. Выявлены классы циклов, которые не поддаются векторизации на последней версии компилятора. Реализован оптимизационный пасс, расширяющий класс векторизуемых циклов.

Ключевые слова: векторизация; циклы; компиляторы; RISC-V.

1. Введение

Многие архитектуры набора команд имеют в своём наборе специальные векторные инструкции, позволяющие выполнять операцию над множеством данных одновременно. Благодаря подобным инструкциям становится возможен параллелизм на уровне инструкций, при котором даже одно процессорное ядро способно параллельно обрабатывать данные, без необходимости в построении архитектуры программы в соответствии с принципами параллельного программирования.

Векторизация – это процесс написания или трансляции программного кода с учётом использования векторных инструкций целевой платформы. Наиболее предпочтительной является именно трансляция, при которой во время оптимизации исходного кода циклы автоматически векторизуются, потому что при таком подходе исходный код программы менее зависит от целевой платформы, а также не теряет свою читаемость. На данный момент библиотека LLVM содержит информацию о векторных инструкциях следующего списка архитектур [6]: AArch64, AMDGPU, ARM, Hexagon, Mips, NVPTX, PowerPC, RISCV, Spark, SPIRV, SystemZ, VE, WebAssembly, X86, Xtensa. При этом векторизатор LLVM, как показало исследование [5], способен автоматически векторизовать 2 трети циклов из тестового набора ETSVC, разработанного специально для проверки способностей компиляторов в области автоматической векторизации кода.

Компилятор clang и другие продукты, разработанные с использованием библиотеки LLVM во время компиляции используют специальное промежуточное представление (LLVM IR), в котором код любой программы не зависит от языка программирования исходного кода, а также от целевой архитектуры. Благодаря этому любые архитектурно независимые оптимизации могут быть применены к программе, вне зависимости от того, на каком языке программирования она была написана изначально и для какой платформы она компилируется. Казалось бы, при этом выполнение векторизации было бы невозможно, ведь у каждой архитектуры свой набор векторных инструкций и векторных регистров, но эту проблему удалось решить посредством того, что в промежуточном представлении есть отдельный тип данных для представления вектора любого размера, и компилятор может учитывать размер векторных регистров (как и факт их наличия) во время компиляции для принятия решения о том, какого размера векторы следует использовать в промежуточном представлении.

2. Обзор проблемы

Автоматическая векторизация позволяет повысить производительность программы без необходимости в модификации исходного кода, но современные компиляторы по-прежнему

не могут векторизовать все случаи, которые теоретически поддаются векторизации. В данной работе были выбраны 2 тестовых случая из набора ETSVC [4], которые не векторизуются компилятором clang, для поиска способов расширить возможности компилятора. При попытке векторизовать циклы в функциях s291 и s292, если указать опцию Rpass-analysis=loop-vectorize для вывода отладочной информации, компилятор выведет сообщение «remark: loop not vectorized: value that could not be identified as reduction is used outside the loop», что указывает на неустранимую зависимость по данным между итерациями. На самом деле, clang способен автоматически разрешать зависимость между итерациями в ходе оптимизации, но конкретно в этом случае ему мешает ситуация, при которой переменная объявленная за пределами цикла и используемая в теле цикла имеет специфичное значение на первой итерации, на зависящее от индукционной переменной. Решение этой проблемы описывается далее.

3. Разработка оптимизационного пасса

Решение описанной выше проблемы сводится к простой идее: если векторизации цикла мешает значение переменной на первой итерации, значит необходимо выполнить сначала первую итерацию цикла отдельно, а затем весь остальной цикл, то есть произвести пилинг (peeling) или отслаивание цикла на одну итерацию. Основную трудность составляет разработка условия, которое позволит разделить все циклы на 2 класса, в одном из которых все циклы, подобные тем, которые исследуются в рамках данной работы, а в другом все остальные циклы.

Так как все оптимизации LLVM происходят над промежуточным представлением, то и при разработке условия необходимо отталкиваться от того, кода, который получается на этапе промежуточного представления. На листинге 1 представлено промежуточное представление цикла из функции s291, подающееся на вход пассу LoopVectorizePass.

```
1 ; Loop:
2 5: ; preds = %0, %5
3 %6 = phi i32 [ 0, %0 ], [ %17, %5 ]
4 %7 = phi i32 [ 31999, %0 ], [ %6, %5 ]
5 %8 = zext nneg i32 %6 to i64
6 %9 = getelementptr inbounds nuw float, ptr @b, i64 %8
7 %10 = load float, ptr %9, align 4, !tbaa !9
8 %11 = zext nneg i32 %7 to i64
9 %12 = getelementptr inbounds nuw float, ptr @b, i64 %11
10 %13 = load float, ptr %12, align 4, !tbaa !9
11 %14 = fadd fast float %13, %10
12 %15 = fmul fast float %14, 5.000000e-01
13 %16 = getelementptr inbounds nuw float, ptr @a, i64 %8
14 store float %15, ptr %16, align 4, !tbaa !9
15 %17 = add nuw nsw i32 %6, 1
16 %18 = icmp samesign ult i32 %6, 31999
17 br i1 %18, label %5, label %2, !llvm.loop !13
```

Листинг 1. Код промежуточного представления цикла s291

4. Разработка оптимизационного пасса

Здесь описывается устройство работы реализованного пасса.

5. Экспериментальная проверка качества пасса

Тут повторные эксперименты и сравнение с предыдущими результатами.

6. Заключение

Литература

1. Moldovanova O., Kurnosov M. Auto-Vectorization of Loops on Intel 64 and Intel Xeon Phi: Analysis and Evaluation // Proc. of the 14th International Conference on Parallel Computing Technologies (PaCT-2017), 2017. – Springer LNCS 10421. – P. 143-150.
2. Maleki S., Gao Ya. Garzarán M.J., Wong T., Padua D.A. An Evaluation of Vectorizing Compilers // Proc. of the International Conference on Parallel Architectures and Compilation Techniques (PACT'11). 2011. P. 372 – 382.
3. Extended Test Suite for Vectorizing Compilers [Электронный ресурс]. URL: <https://github.com/PGG106/TSVC> (дата обращения 07.04.2025).
4. Extended Test Suite for Vectorizing Compilers (Fixed) [Электронный ресурс]. URL: https://github.com/UoB-NPC/TSVC_2 (дата обращения 07.04.2025).
5. Кулик П. Е. Анализ эффективности автоматической векторизации циклов на архитектуре RISC-V RV64GCVB // Обработка информации и математическое моделирование. Материалы Всероссийской научно-практической конференции, 24 апреля 2025 г. / Сибирский государственный университет телекоммуникаций и информатики ; под редакцией И. О. Орлова, В. И. Ефановой, – Новосибирск: СибГУТИ, 2025. – Текст: электронный. – 2025. – 436 с. ISBN 978-5-91434-125-8 С. 111–116.
6. LLVM Project [Электронный ресурс]. URL: <https://github.com/llvm/llvm-project> (дата обращения 10.11.2025)

Кулик Павел Евгеньевич

студент 4 курса института информатики и вычислительной техники СибГУТИ, (630102, Новосибирск, ул. Кирова, 86), тел. +7 923 462-60-30, e-mail: pashakulik58@gmail.com.

Курносов Михаил Георгиевич (научный руководитель)

профессор кафедры вычислительных систем СибГУТИ (630102, Новосибирск, ул. Кирова, 86), тел. (383) 269-82-75, e-mail: mkurnosov@gmail.com

Kulik P. E., Kurnosov M. G. (scientific adviser)

Expanding the capabilities of the clang vectorizer by implementing an optimization pass

The ability of automatic cycle vectorization from the Extended Test Suite For Vectorizing Compilers (ETSVC) with the Clang/LLVM compiler for the RISC-V RV64GCVB architecture has been analyzed. Classes of loops that cannot be vectorized on the latest version of the compiler have been identified. An optimization pass has been implemented that expands the class of vectorizable cycles.

Keywords: vectorization; loops; compilers; RISC-V.